

Stage 1 — Python, Git, Venv & Local Model Setup

Sprint: Sprint 1

Status: Complete

Goal

Set up a clean local development environment and verify that Python can use our local AI models through Ollama.

What we set up

- Python 3.11
- VS Code
- Git + GitHub
- dev branch for active development
- .venv for isolated Python packages
- pyproject.toml for project dependencies
- Ollama as the local model runtime

Models

We installed:

qwen3.5:4b

embeddinggemma

Qwen is used for generating text.

EmbeddingGemma is used for turning text into embedding vectors, which will later be used for RAG retrieval.

Python + Ollama

We installed the Ollama Python client:

```
python -m pip install ollama
```

and added it to pyproject.toml:

```
dependencies = [  
    "ollama>=0.6.2",
```

]

The Python package lets our code communicate with the Ollama server running locally.

The basic flow is:

Python code

↓

Ollama Python client

↓

Ollama

↓

Local model

Tests

We verified both model paths successfully:

Python → Ollama → Qwen → text response

and:

Python → Ollama → EmbeddingGemma → embedding vector

We then created a reusable smoke-test script inside:

backend/app/scripts/

and confirmed that it runs successfully from the project environment.

Git

The setup and smoke tests were committed and pushed to the dev branch.

Local-only files such as .venv, model files, personal data, databases, and secrets are not meant to be stored in Git.

Result

At the end of Stage 1, we have a working local AI foundation:

Python

↓

Ollama

└─ Qwen3.5 → text generation

└─ EmbeddingGemma → embeddings

The environment is ready for Sprint 2, where we start building the first working RAG pipeline.